

Project 2 - Cybersecurity Analytics

K-Nearest Neighbor (KNN) Classifier for Network Intrusion Detection

Introduction

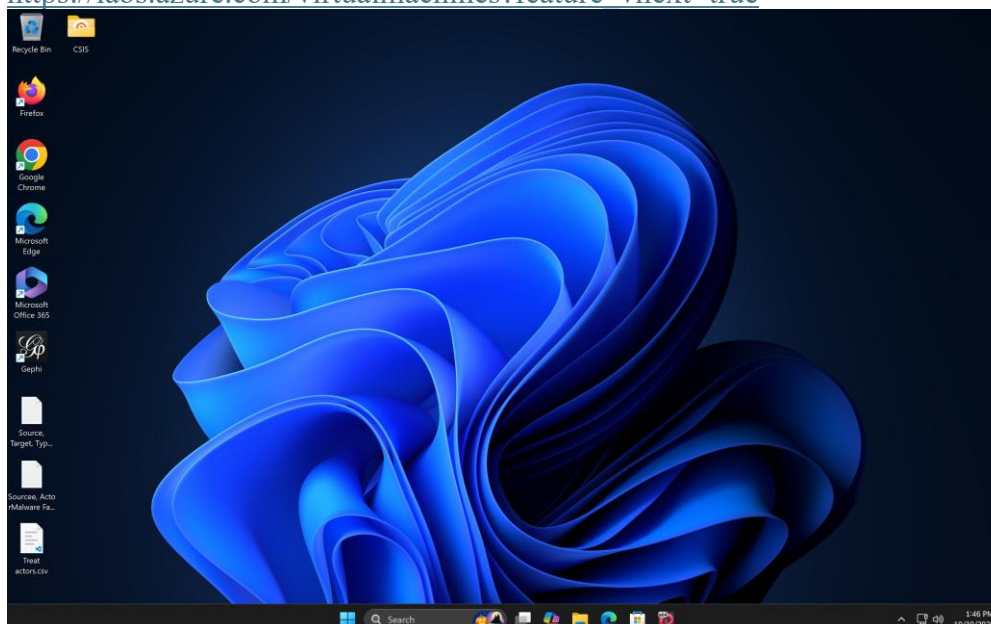
This lab is designed to work with CSIS-Admin VM machine to get practical hands-on experience in machine learning–driven security analytics. Today, machine learning plays a major role in cybersecurity because it helps detect anomalies, classify malicious network behavior, and support automated threat responses. As cyberattacks continue to grow rapidly, machine learning models like K-Nearest Neighbor (KNN) are becoming important tools to improve intrusion detection accuracy and help organizations make better security decisions based on data.

According to Sotiropoulos (2024), adversarial AI attacks including evasion, poisoning, and model extraction can create serious risks by exploiting how machine learning algorithms work internally. Attackers may purposely craft inputs to evade detection, reduce model confidence, or manipulate a model's output, especially in sensitive environments like network intrusion detection systems. Chio and Freeman (2018) also explain that machine learning models used in security often inherit weaknesses from the data they are trained on, which makes it important to analyze and visualize how these models behave.

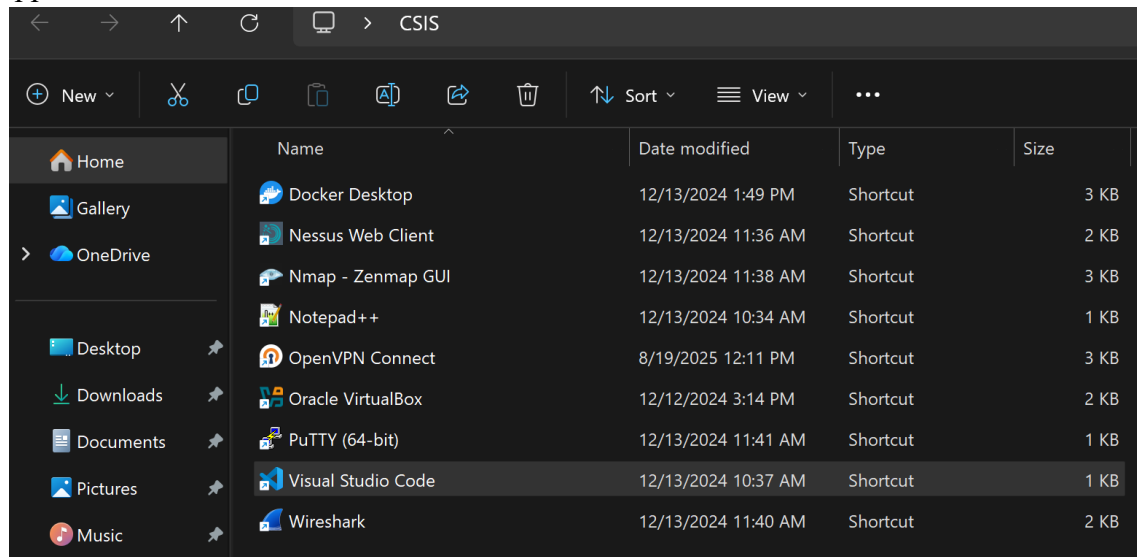
In this lab, the KNN classifier is used on a KDD-based dataset (lab_data_kdd_c.csv). First, the required packages are imported, and the dataset is prepared by encoding categorical values and selecting training and testing samples. Then, a KNN classification model is trained, and evaluation metrics such as the confusion matrix, accuracy, and error rate are generated to better understand how the model performs. Finally, a contour plot is created to visualize the decision boundary of the KNN classifier. Overall, this lab provides a complete workflow for understanding how KNN can be applied in cybersecurity analytics and how its results can be interpreted.

1. Logged to Azure Lab environment using following web link and logged into the Kali VM using Oracle VirtualBox.

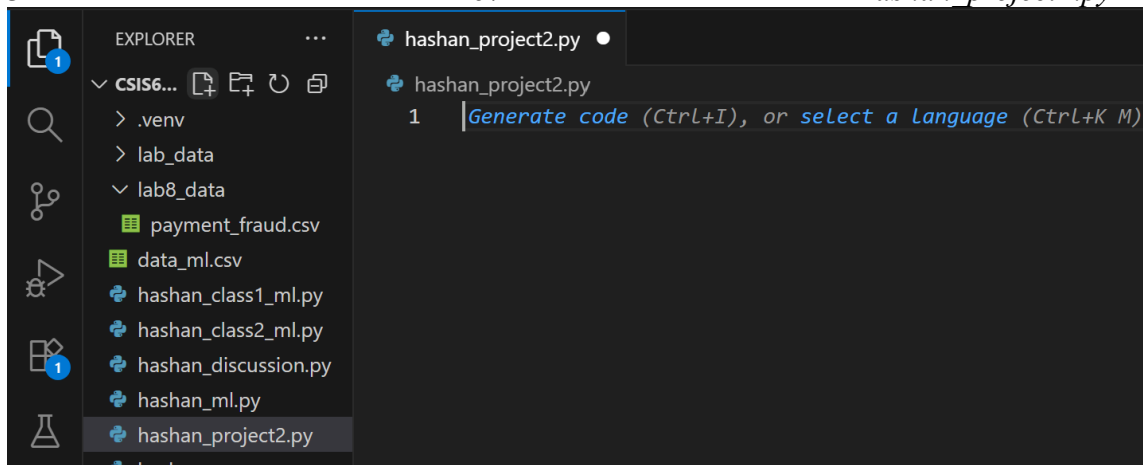
https://labs.azure.com/virtualmachines?feature_vnext=true



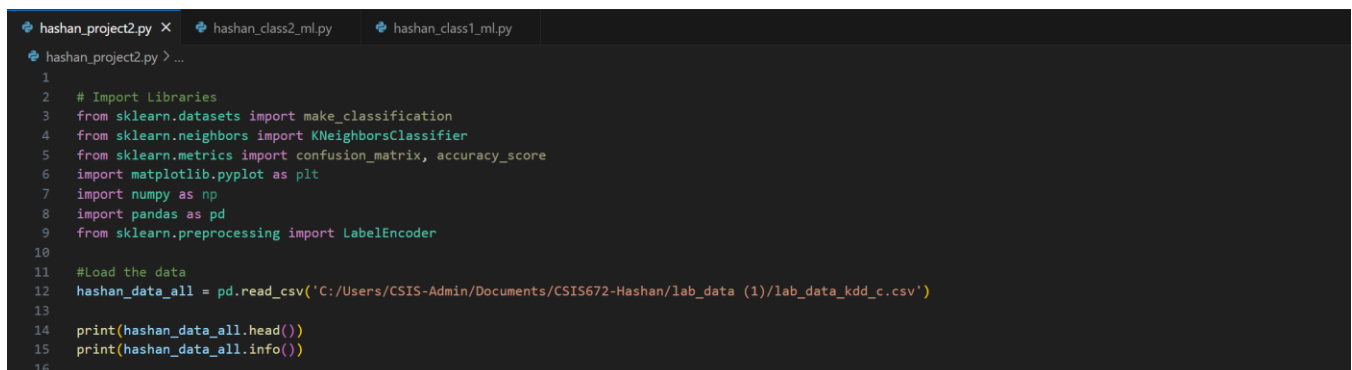
2. Located VS Code under the folder CSIS (C:\Users\CSIS-Admin\Desktop\CSIS) and run the application.



3. Created new file under CSIS672-HASHAN and named as *Hashan_project2.py*



4. Imported Python libraries to support data loading, visualization, and classification using the K-Nearest Neighbor (KNN) algorithm. Then, loaded the provided dataset *lab_data_kdd_c.csv* and printed the head and info of dataset.



Output

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(.venv) PS C:\Users\CSIS-Admin\Documents\CSIS672-Hashan> & C:\Users\CSIS-Admin\Documents\CSIS672-Hashan\.venv\Scripts\python.exe c:\Users\CSIS-Admin\Documents\CSIS672-Hashan\hashan_project2.py
  protocol_type  service flag  land  logged_in  is_host_login  ...  dst_host_srv_diff_host_rate  dst_host_serror_rate  dst_host_srv_serror_rate  dst_host_rerror_rate  dst_host_srv_rerror_rate  category
0      tcp      ftp_data  SF      0          0          0          ...          0.00          0.00          0.00          0.00          0.05          0.00  benign
1      udp      other     SF      0          0          0          ...          0.00          0.00          0.00          0.00          0.00          0.00  benign
2      tcp      private  S0      0          0          0          ...          0.00          1.00          1.00          0.00          0.00          0.00  dos
3      tcp      http     SF      0          1          0          ...          0.04          0.03          0.01          0.00          0.00          0.01  benign
4      tcp      http     SF      0          1          0          ...          0.00          0.00          0.00          0.00          0.00          0.00  benign

[5 rows x 42 columns]
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 148517 entries, 0 to 148516
Data columns (total 42 columns):
#   Column              Non-Null Count  Dtype
---  -
0   protocol_type        148517 non-null  object
1   service              148517 non-null  object
2   flag                 148517 non-null  object
3   land                 148517 non-null  int64
4   logged_in            148517 non-null  int64
5   is_host_login        148517 non-null  int64
6   is_guest_login       148517 non-null  int64
7   duration             148517 non-null  int64
8   src_bytes            148517 non-null  int64
9   dst_bytes            148517 non-null  int64
10  wrong_fragment       148517 non-null  int64
11  urgent               148517 non-null  int64

```

The lab_data_kdd_c.csv file is a KDD-based intrusion detection dataset that contains a mixture of numerical and categorical network traffic features. After loading the dataset, the `.head()` and `.info()` outputs confirmed that the dataset includes 148,517 rows and 42 feature columns. These features represent different characteristics of network connections, including:

Numerical features- such as `src_bytes`, `dst_bytes`, `duration`, `wrong_fragment`, `hot`, `num_failed_logins`, `count`, `srv_count`, and several host-level rate metrics (ex: `dst_host_serror_rate`, `dst_host_rerror_rate`).

Categorical features;

- `protocol_type` - tcp, udp, icmp
- `service` - http, ftp, smtp
- `flag` - SF, S0
- `category` - benign (legitimate network traffic), dos (denial-of-service attacks), probe (scanning or probing attacks), r2l (remote-to-local attacks), u2r (user-to-root privilege escalation).

The KNN requires numerical values for classification, all categorical columns were later encoded into integers.

5. The Dataset preprocessed and split for machine-learning classification. All categorical attributes encoded into numerical values using LabelEncoder to make the dataset compatible with the KNN algorithm. Then the encoded dataset divided into 100000 training samples and 35000 testing samples. The KNN classifier (k=5) trained on the prepared training data.

```

hashan_project2.py > ...
17 # Encode categorical columns
18 df_encoded = hashan_data_all.copy()
19 encoders = {}
20
21 for col in ['protocol_type', 'service', 'flag', 'category']:
22     le = LabelEncoder()
23     df_encoded[col] = le.fit_transform(hashan_data_all[col])
24     encoders[col] = dict(zip(le.classes_, le.transform(le.classes_)))
25     print(f"\n{col} mapping:", encoders[col])
26
27 # Define features and target
28 X = df_encoded.drop('category', axis=1)
29 y = df_encoded['category']
30
31 # Specify train and test sizes
32 n_train = 100000
33 n_test = 35000
34
35 # Create training and testing datasets
36 X_train = X[:n_train]
37 y_train = y[:n_train]
38
39 X_test = X[n_train:n_train+n_test]
40 y_test = y[n_train:n_train+n_test]
41
42 # Train the KNN classifier
43 clf = KNeighborsClassifier(n_neighbors=5).fit(X_train, y_train)
44
45 print("KNN model successfully trained on", n_train, "samples.")
46

```

Output

```

(venv) PS C:\Users\CSIS-Admin\Documents\CSIS672-Hashan> & C:/Users/CSIS-Admin/Documents/CSIS672-Hashan/.venv/Scripts/python.exe c:/Users/CSIS-Admin/Documents/CSIS672-Hashan/hashan_project2.py

protocol_type mapping: {'icmp': np.int64(0), 'tcp': np.int64(1), 'udp': np.int64(2)}

service mapping: {'IRC': np.int64(0), 'X11': np.int64(1), 'Z39_50': np.int64(2), 'aol': np.int64(3), 'auth': np.int64(4), 'bgp': np.int64(5), 'courier': np.int64(6), 'csnet_ns': np.int64(7), 'ctf': np.int64(8), 'daytime': np.int64(9), 'discard': np.int64(10), 'domain': np.int64(11), 'domain_u': np.int64(12), 'echo': np.int64(13), 'eco_i': np.int64(14), 'ecr_i': np.int64(15), 'efs': np.int64(16), 'exec': np.int64(17), 'finger': np.int64(18), 'ftp': np.int64(19), 'ftp_data': np.int64(20), 'gopher': np.int64(21), 'harvest': np.int64(22), 'hostnames': np.int64(23), 'http': np.int64(24), 'http_2784': np.int64(25), 'http_443': np.int64(26), 'http_8001': np.int64(27), 'imap4': np.int64(28), 'iso_tsap': np.int64(29), 'klogin': np.int64(30), 'kshell': np.int64(31), 'ldap': np.int64(32), 'link': np.int64(33), 'logi_n': np.int64(34), 'ntp': np.int64(35), 'name': np.int64(36), 'netbios_dgm': np.int64(37), 'netbios_ns': np.int64(38), 'netbios_sn': np.int64(39), 'netstat': np.int64(40), 'nmap': np.int64(41), 'ntp': np.int64(42), 'ntp_u': np.int64(43), 'other': np.int64(44), 'pm_dump': np.int64(45), 'pop_2': np.int64(46), 'pop_3': np.int64(47), 'printer': np.int64(48), 'private': np.int64(49), 'red_i': np.int64(50), 'remote_job': np.int64(51), 'rje': np.int64(52), 'shell': np.int64(53), 'smtp': np.int64(54), 'sql_net': np.int64(55), 'ssh': np.int64(56), 'sunrpc': np.int64(57), 'supdup': np.int64(58), 'systat': np.int64(59), 'telnet': np.int64(60), 'tftp_u': np.int64(61), 'tim_i': np.int64(62), 'time': np.int64(63), 'urh_i': np.int64(64), 'urp_i': np.int64(65), 'uucp': np.int64(66), 'uucp_path': np.int64(67), 'vmnet': np.int64(68), 'whois': np.int64(69)}

flag mapping: {'OTH': np.int64(0), 'REJ': np.int64(1), 'RSTO': np.int64(2), 'RSTOS0': np.int64(3), 'RSTR': np.int64(4), 'S0': np.int64(5), 'S1': np.int64(6), 'S2': np.int64(7), 'S3': np.int64(8), 'SF': np.int64(9), 'SH': np.int64(10)}

category mapping: {'benign': np.int64(0), 'dos': np.int64(1), 'probe': np.int64(2), 'r21': np.int64(3), 'ur2': np.int64(4)}

KNN model successfully trained on 750 samples.
(venv) PS C:\Users\CSIS-Admin\Documents\CSIS672-Hashan>

```

The code applies label encoding to each categorical feature. *fit_transform()* converts string values into integer labels required for KNN classification. A dictionary of the encoded mappings is generated using *le.classes_* and *le.transform()*, and this mapping is printed for reference. This ensures transparency in how categorical features such as protocol type, service type, flag, and attack category are converted into numeric forms.

6. First, generated predictions for testing dataset and then created commands to calculate Confusion matrix, Accuracy, and Error rate.

```
47 # Generate predictions for the testing dataset
48 y_pred = clf.predict(X[n_train:n_train+n_test])
49
50 # Compute confusion matrix and accuracy/error rate
51 cm = confusion_matrix(y[n_train:n_train+n_test], y_pred)
52 accuracy = accuracy_score(y[n_train:n_train+n_test], y_pred)
53 error_rate = 1 - accuracy
54
```

Created print commands to capture the results. Then, the commands executed to capture the output.

```
50 # Compute confusion matrix and accuracy/error rate
51 cm = confusion_matrix(y[n_train:n_train+n_test], y_pred)
52 accuracy = accuracy_score(y[n_train:n_train+n_test], y_pred)
53 error_rate = 1 - accuracy
54
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(.venv) PS C:\Users\CSIS-Admin\Documents\CSIS672-Hashan> & C:/Users/CSIS-Admin/Documents/CSIS672-Hashan/.
y
KNN model successfully trained on 100000 samples.
Confusion Matrix: [[17552  69  90  7  1]
 [ 697 11672  200  0  0]
 [ 222  279 2876  0  0]
 [ 1007  2  33 203  0]
 [  41  16  29  0  4]]
Accuracy: 0.9230571428571429
Error Rate: 0.07694285714285709
```

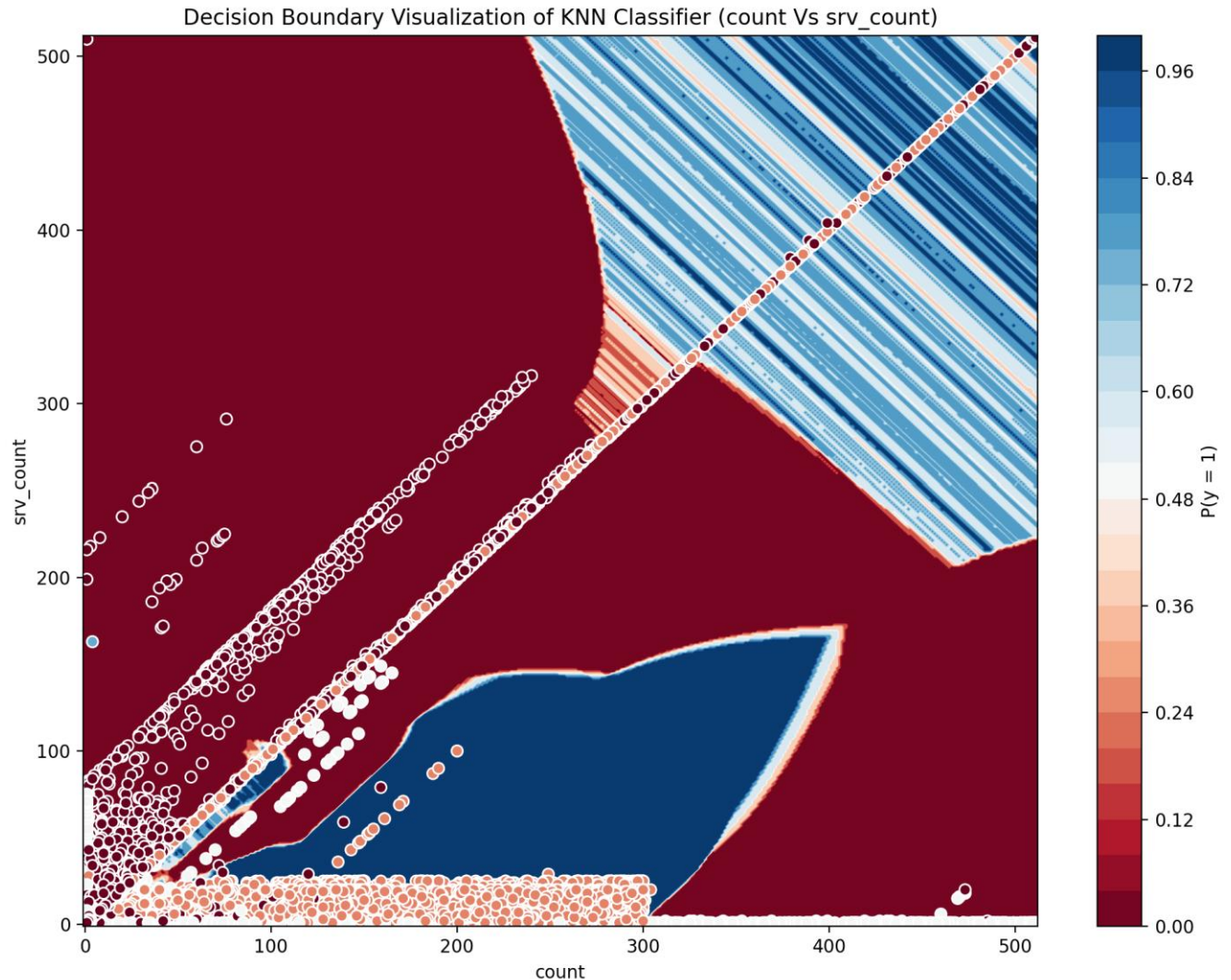
The predictions from the KNN classifier were compared to the true labels of the testing dataset. A confusion matrix was generated to summarize true positives, true negatives, false positives, and false negatives. The accuracy and error rate were computed to evaluate overall classifier performance.

The evaluation showed an overall model accuracy of 92.3%, with an error rate of 7.7%. The confusion matrix revealed strong performance on “benign” and “DoS” traffic, while the “probe” category showed slightly higher misclassification rates, likely due to feature overlap.

8. To visualize how the KNN model separates different classes, a decision boundary contour plot generated using two numeric features from the dataset. The lab dataset contains many high-range numeric fields, and I chose the *count* and *srv_count* features to get meaningful variation and represent network-level behavior.

```
60 #-----
61
62 # Select two numeric features for visualization
63 feature1 = 'count'
64 feature2 = 'srv_count'
65
66 X_vis = df_encoded[[feature1, feature2]].values
67
68 # Re-train KNN classifier using only these 2 features
69 clf_vis = KNeighborsClassifier(n_neighbors=5).fit(X_vis[:n_train], y_train)
70
71 # Create meshgrid for visualization
72 x_min, x_max = X_vis[:, 0].min() - 1, X_vis[:, 0].max() + 1
73 y_min, y_max = X_vis[:, 1].min() - 1, X_vis[:, 1].max() + 1
74
75 xx, yy = np.meshgrid(
76     np.linspace(x_min, x_max, 400),
77     np.linspace(y_min, y_max, 400)
78 )
79
80 grid = np.c_[xx.ravel(), yy.ravel()]
81
82 # Predict class probabilities for the grid
83 probs = clf_vis.predict_proba(grid)[:, 1].reshape(xx.shape)
84
85 # Plot the decision boundary
86 plt.figure(figsize=(12, 9))
87 contour = plt.contourf(xx, yy, probs, 25, cmap="RdBu", vmin=0, vmax=1)
88 plt.colorbar(contour).set_label("P(y = 1)")
89
90 # Plot the test set
91 plt.scatter(X_vis[n_train:n_train+n_test, 0],
92             X_vis[n_train:n_train+n_test, 1],
93             c=y_test,
94             cmap="RdBu",
95             s=40,
96             edgecolor="white")
97
98 plt.title("Decision Boundary Visualization of KNN Classifier (count Vs srv_count)")
99 plt.xlabel(feature1)
100 plt.ylabel(feature2)
101 plt.show()
```


Output



- The contour regions represent how the KNN classifier assigns probability to class labels based on the two selected numeric features (*count* and *srv_count*).
- Red regions indicate higher predicted probability for an attack category.
- Blue regions represent zones where the model predicts benign traffic.
- The white circular markers are the actual test samples from the dataset, colored by their true category label.
- The diagonal striped patterns indicate strongly correlated relationships between *count* and *srv_count* for certain attack types.
- The jagged and non-linear boundary edges reflect how KNN classification decisions depend on the local neighbors around each data point.

This figure demonstrates that KNN can form meaningful separation boundaries even with only two features. When we consider network traffic clusters,

- Benign traffic appears mostly in the lower-left region where both count and srv_count are low.
- Attack traffic forms dense clusters along diagonal lines. These likely represent systematic probing or scanning behaviors in the dataset.
- The large red zones indicate broad areas where the classifier strongly predicts attacks.
- The blue central region suggests some separation between benign and attack traffic, even with only two features.

9. The KNN classification model was evaluated using confusion matrix, accuracy, and error rate. The results indicate a strong overall performance, especially considering the dataset includes multiple attack categories and noisy behaviors. When we consider the performance, it was given 92.7% accuracy and 7.3% error rate. The confusion matrix revealed strong performance on “benign” and “DoS” traffic, while the “probe” category showed slightly higher misclassification rates, likely due to feature overlap. Also, it recognized rare categories such as R2L/U2R. These results suggest that KNN is effective for network intrusion detection when provided with properly encoded features.

Confusion matrix output shows, *benign* traffic (label 1) was classified correctly 17552 times, with only minor spillovers into attack categories. DoS traffic (label 2) showed consistently high accuracy with almost no misclassification. It correctly identified 11,672 times, with limited confusion into benign and probe categories. Probe attacks (label 3) were recognized correctly 203 times but misclassified heavily into benign and probe categories due to their low representation. U2R attacks (label 4) were also detected but had very small sample size, which limits generalization.

The KNN classifier achieved high accuracy, generated meaningful decision boundaries, and successfully distinguished key traffic categories in the given dataset. Also, KNN is a viable baseline model for intrusion detection, especially when paired with proper preprocessing and visualization.

Conclusion

This lab helped me practically understand how machine-learning models can be used in cybersecurity, especially for intrusion detection. By using the KNN classifier on the KDD-based dataset, I was able to see how data preprocessing, encoding, training, and testing all work together in a full classification workflow. The evaluation results showed that the KNN model performed well with a 92.7% accuracy, and the confusion matrix helped highlight which attack categories were correctly detected and where misclassifications happened.

The decision boundary visualization also gave a clear idea of how KNN separates different traffic patterns using numerical features. This lab clearly demonstrates that KNN can be a useful baseline model for network intrusion detection when proper preprocessing is applied. It also gave me a better understanding of how machine learning can support security analytics and how decision boundaries and confusion matrices help interpret the behavior of the model.

References

1. Chio, C., & Freeman, D. (2018). *Machine Learning and Security: Protecting Systems with Data and Algorithms*. O'Reilly Media.
2. Sotiropoulos, J. (2024). *Adversarial AI attacks, mitigations, and defense strategies*. Packet Publishing.
3. GeeksforGeeks. (2025, July 11). *Pandas Read CSV in Python*. Sanchhaya Education Private Limited. https://www.geeksforgeeks.org/pandas/python-read-csv-using-pandas-read_csv/
4. Microsoft Docs. (2025). *Python in Visual Studio Code*. Microsoft. Retrieved November 30, 2025, from <https://code.visualstudio.com/docs/languages/python>
5. McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media
6. Minnesota State University Moorhead. (2025). Course content: Cybersecurity Analytics. D2L Brightspace. Retrieved November 30, 2025 from <https://mnstate.learn.minnstate.edu/d2l/le/content/7131515/Home>